APACHE SPARK

Why Spark?

Spark is a distributed data processing framework. Instead of making one computer process a very large dataset alone, Spark divides the data into smaller pieces and processes those pieces in parallel across multiple machines.

IT takes the user's PySpark or SQL request, creates and analyzes a logical plan, optimizes it using Catalyst, creates a physical execution plan, divides the work into stages and tasks, and distributes those tasks across executors to process data in parallel.

1. Architecture Flow

```
USER
↓
PySpark / Spark SQL
↓
DRIVER
↓
Spark DataFrame API / Spark SQL
↓
Unresolved Logical Plan
↓
Analyzer
↓
Resolved Logical Plan
↓
Catalyst Optimizer
↓
Optimized Logical Plan
↓
Physical Planning
↓
Physical Plan
↓
DAG / Scheduling
↓
STAGES
↓
TASKS
↓
EXECUTORS
↓
RESULT
```

3. Driver and Executors

Driver

The Driver is the main coordinator of a Spark application

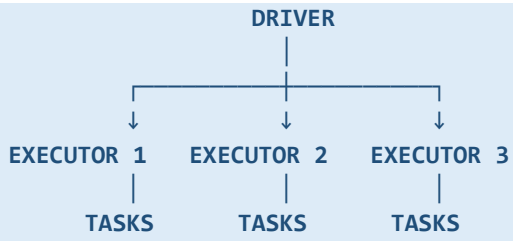
- Receives the Spark application
- Builds the execution plan
- Creates the DAG
- Divides work into stages and tasks
- Schedules tasks
- Coordinates executors
- Tracks execution and handles results

```
USER CODE → DRIVER → PLANS / SCHEDULING → EXECUTORS
```

Executors

Executors are worker processes.

They perform the actual tasks assigned by the Driver and can keep cached data for reuse.



Driver = coordinator.

Executors = workers.

4. Partitions

A partition is a logical piece of a dataset.

Partitions are the pieces of data on which tasks perform work.

Spark divides a large dataset into partitions so different pieces can be processed in parallel.

```
100 GB DATA → Partition 1 | Partition 2 | Partition 3 | ... → Parallel processing
```

6. Tasks

A task is a unit of work performed by an executor on one partition for a particular stage.

```
STAGE → PARTITIONS → TASKS → EXECUTORS
```

For a stage, Spark generally creates one task for each partition that the stage processes.

7. Transformations and Actions

Transformation

A transformation describes how data should be changed. Transformations are normally evaluated lazily.

```
df2 = df.filter(df.age > 25)
df3 = df2.select("name", "age")
```

Examples: filter(), select(), withColumn(), groupBy(), join().

Action

An action requests a result and triggers the execution of the required transformations.

```
df3.show()
df3.count()
```

Examples: show(), count(), collect(), first(), and writing output.

8. Lazy Evaluation

Lazy evaluation means Spark does not immediately execute every transformation when the code is written. Spark waits until an action requires a result.

```
TRANSFORMATIONS → BUILD PLAN → ACTION → EXECUTION
```

use: Spark can look at the complete chain of requested operations and optimize the work before executing it.

9. DataFrame API

The DataFrame API lets us describe data processing using programming operations on DataFrames.

```
result = (
    df.filter(df.amount > 1000)
      .select("customer", "amount")
)
result.show()
```

Common operations: select(), filter(), where(), withColumn(), drop(), groupBy(), agg(), join(), orderBy().

10. Spark SQL

Spark SQL lets us describe Spark data processing using SQL.

```
df.createOrReplaceTempView("sales")

result = spark.sql("""
    SELECT city, SUM(amount)
    FROM sales
    WHERE amount > 1000
    GROUP BY city
""")
```

NOTE: DataFrame API and Spark SQL are different ways of expressing a data-processing request. They use Spark's execution machinery.

11. Logical Plan

A logical plan describes what operations Spark needs to perform, without deciding exactly how those operations will run on the cluster.

```
Read → Filter → Group By → Aggregate
```

12. Unresolved Logical Plan

At first, Spark may not know exactly what every table, column, or data type refers to.

This initial representation is the unresolved logical plan.

```
USER CODE → UNRESOLVED LOGICAL PLAN
```

13. Analyzer

The Analyzer resolves names and references in the logical plan. It checks things such as column names, table names, and data types.

```
UNRESOLVED LOGICAL PLAN → ANALYZER → RESOLVED LOGICAL PLAN
```

Example: If the query uses amount, Spark determines which amount column is meant and what its data type is.

14. Resolved Logical Plan

After analysis, Spark has a logical plan where the required references have been resolved.

```
Read sales → Filter sales.amount > 1000 → Group by sales.city → Sum sales.amount
```

15. Catalyst Optimizer

Catalyst is Spark SQL's query optimization framework. It examines the resolved plan and tries to produce a more efficient plan.

```
RESOLVED LOGICAL PLAN → CATALYST OPTIMIZER → OPTIMIZED LOGICAL PLAN
```

Common optimization ideas include:

- **Predicate pushdown**: move filtering closer to the data source when possible.
- **Column pruning:** avoid reading or processing columns that are not needed.
- **Expression simplification:** simplify calculations or conditions when possible.
- Other rule-based and cost-based improvements used by Spark's query engine.

16. Predicate Pushdown

If the data source supports it, Spark may push a filter closer to the source so less data has to be read or processed.(Filter as early as possible when the source and query allow it.)

```
READ ALL DATA → FILTER

Better when supported:
DATA SOURCE → FILTER EARLY → LESS DATA → FURTHER PROCESSING
```

17. Column Pruning

If a dataset contains many columns but the query needs only a few, Spark may avoid reading or carrying unnecessary columns when supported.

```
MANY COLUMNS → NEED ONLY city + salary → READ / PROCESS FEWER COLUMNS
```

18. Physical Plan

The physical plan describes the execution strategies Spark will actually use to perform the logical operations.

The physical plan is much closer to the actual execution work than the logical plan.

```
LOGICAL PLAN = WHAT
PHYSICAL PLAN = HOW

File Scan → Filter → Exchange / Shuffle → Aggregation
```

19. Complete Planning Flow

```
User Code
    ↓
Unresolved Logical Plan
    ↓
Analyzer
    ↓
Resolved Logical Plan
    ↓
Catalyst Optimizer
    ↓
Optimized Logical Plan
    ↓
Physical Planning
    ↓
Physical Plan
```

20. DAG — Directed Acyclic Graph

DAG stands for **Directed Acyclic Graph**.

In Spark, the DAG represents dependencies between operations in the execution plan.

```
Read → Filter → Select → Group By → Sum
```

Directed means the dependency has a direction.

Acyclic means it does not contain a cycle.

Graph means connected operations are represented as a structure.

21. Shuffle

Shuffle is the redistribution of data between partitions so that related records can be brought together.

Example: groupBy("city") may require records for the same city to be brought together.

NOTE: Shuffle can involve network communication and intermediate data movement, so it is generally an expensive part of distributed processing.

22. Narrow and Wide Transformations

Narrow Transformation

A narrow transformation can process an input partition without needing data from many other partitions.

```
df.filter(...)
df.select(...)
df.withColumn(...)
```

```
Partition 1 → Partition 1
Partition 2 → Partition 2
Partition 3 → Partition 3
```

Wide Transformation

A wide transformation requires data to be redistributed across partitions, causing a shuffle.

```
df.groupBy(...)
df.join(...)
df.distinct(...)
df.orderBy(...)
df.repartition(...)
```

```
P1 ─┐
P2 ─┼→ SHUFFLE → New partitions
P3 ─┤
P4 ─┘
```

23. Stage

A stage is a group of tasks that can execute together without crossing a shuffle boundary.

A shuffle boundary generally separates stages.

```
STAGE 1
Read → Filter → Select
   ↓
SHUFFLE
   ↓
STAGE 2
Group By → Sum
```

24. DAG → Stage → Task → Executor

```
DAG
↓
STAGES
↓
TASKS
↓
EXECUTORS
↓
PROCESS PARTITIONS
↓
RESULT

STAGE 1
├─ Task 1 → Executor 1
├─ Task 2 → Executor 2
├─ Task 3 → Executor 3
└─ Task 4 → Executor 1
```

25. Caching

Caching stores computed data so that it can be reused by later operations instead of recomputing the same lineage.(Cache => keep useful computed data for reuse.)

```
FILTER → CACHE → Count / GroupBy / Other Reuse
```

```
filtered = df.filter(df.amount > 1000)

filtered.cache()

filtered.count()


filtered.groupBy("city").count().show()
```

Cached partitions are associated with executor-side storage.

The Driver coordinates the application; executors process and store the cached partitions.

26. Cache and Lazy Evaluation

Calling cache() marks the DataFrame for caching.

The data is normally computed and cached when an action needs it.

```
df.cache() → still lazy → ACTION → compute + store → later actions can reuse
```

27. Repartition

repartition() changes the number of partitions by redistributing data.

This normally involves a shuffle.

```
df = df.repartition(8)
```

```
4 PARTITIONS → SHUFFLE / REDISTRIBUTE → 8 PARTITIONS
```

It can also repartition based on a column:

```
df = df.repartition(10, "city")
```

28. Coalesce

coalesce() reduces the number of partitions and generally avoids a full shuffle.

(Coalesce = reduce the number of partitions.)

```
df = df.coalesce(4)
```

```
20 PARTITIONS → COALESCE → 4 PARTITIONS
```

29. Repartition vs Coalesce

Feature	Repartition	Coalesce
Changes partition count	Yes	Yes
Increase partitions	Yes	No
Reduce partitions	Yes	Yes
Full shuffle	Normally yes	Normally avoided
Main purpose	Redistribute data	Reduce partitions

30. Join

A join combines two datasets using matching columns.

```
result = orders.join(customers, "customer_id")
```

```
ORDERS + CUSTOMERS → MATCH customer_id → JOINED DATA
```

A normal join may require data redistribution.

The exact physical strategy depends on the data and Spark's chosen plan.

31. Shuffle Join

In a shuffle-based join, Spark redistributes records so matching join keys are brought together.

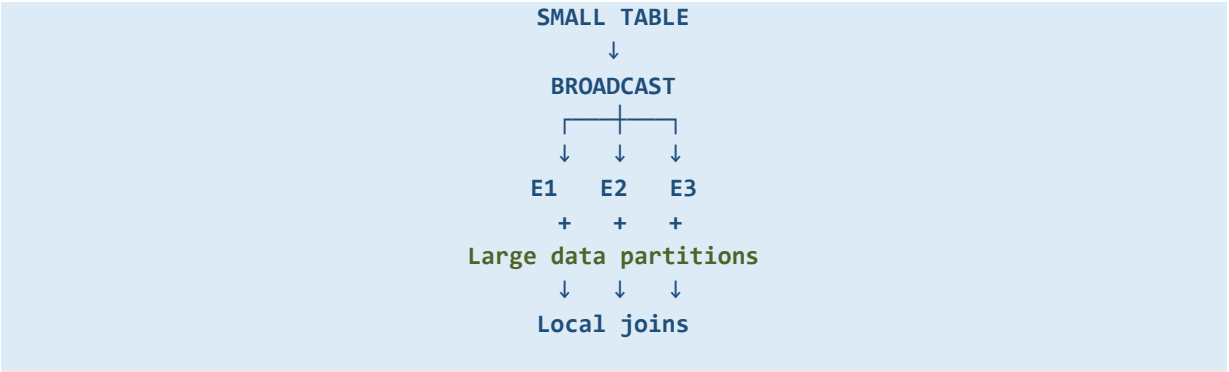
```
ORDERS + CUSTOMERS → SHUFFLE → MATCHING KEYS TOGETHER → JOIN
```

32. Broadcast Join

When one side of a join is small enough, Spark can broadcast that small dataset to executors. Each executor can then join its local large-data partitions with the broadcast data.

```
from pyspark.sql.functions import broadcast

result = orders.join(
    broadcast(customers),
    "customer_id"
)
```



Broadcast join => send the small table to executors so a large-table shuffle can often be avoided.

33. When to Use Broadcast Join

Broadcast join is useful when one side of the join is small enough to be safely replicated to the executors.

34. Cache vs Broadcast

Concept	Purpose	Main benefit
Cache	Reuse computed data	Avoid recomputation
Broadcast Join	Distribute a small join side	Can avoid large join shuffle


USER
↓
DRIVER
↓
DataFrame API / Spark SQL
↓
LOGICAL PLAN
↓
ANALYZER
↓
RESOLVED PLAN
↓
CATALYST OPTIMIZER
↓
OPTIMIZED PLAN
↓
PHYSICAL PLAN
↓
DAG
↓
STAGES
↓
TASKS
↓
EXECUTORS
↓
PROCESS PARTITIONS
↓
RESULT
CACHE
→ reuse computed data
REPARTITION
→ redistribute partitions, normally with shuffle
COALESCE
→ reduce partitions, generally avoiding full shuffle
BROADCAST JOIN
→ send small join side to executors
WIDE OPERATIONS
→ often cause shuffle
→ shuffle generally creates stage boundaries

35. Example — Total Sales by City

Assume we have a large sales Parquet dataset and want total sales by city for records where amount is greater than 1000.

```
sales = spark.read.parquet("sales")

filtered = sales.filter(sales.amount > 1000)

result = (
    filtered
    .groupBy("city")
    .sum("amount")
)

result.show()
```

Step 1 — User: The user writes the PySpark code.

Step 2 — Driver: The Driver receives and coordinates the application.

Step 3 — Logical plan: Spark understands Read → Filter → Group By → Sum.

Step 4 — Analyzer: Spark resolves sales.amount, sales.city, and their types.

Step 5 — Catalyst Optimizer: Spark looks for safe opportunities to make the plan more efficient.

Step 6 — Physical plan: Spark selects actual execution strategies, which may include a scan, filter, shuffle/exchange, and aggregation.

Step 7 — DAG: The dependencies between execution operations are represented.

Step 8 — Stages: The shuffle between filtering and aggregation creates a stage boundary.

Step 9 — Tasks: Each stage is divided into tasks based on the partitions it processes.

Step 10 — Executors: Executors run those tasks on their assigned partitions.

Step 11 — Shuffle: Records with the same city can be brought together.

Step 12 — Aggregation: Spark calculates the sum for each city.

Step 13 — Result: The result is returned to the user or written to an output.